

第 18, 19 回 動的計画法

執筆: 安永憲司

これまでに分割統治法やグラフ探索などのアルゴリズム設計法を見てきた。ここでは、部分問題の解を再利用することで問題を効率よく解く**動的計画法** (*dynamic programming*) を紹介する。

1 再考：DAG における最短パス

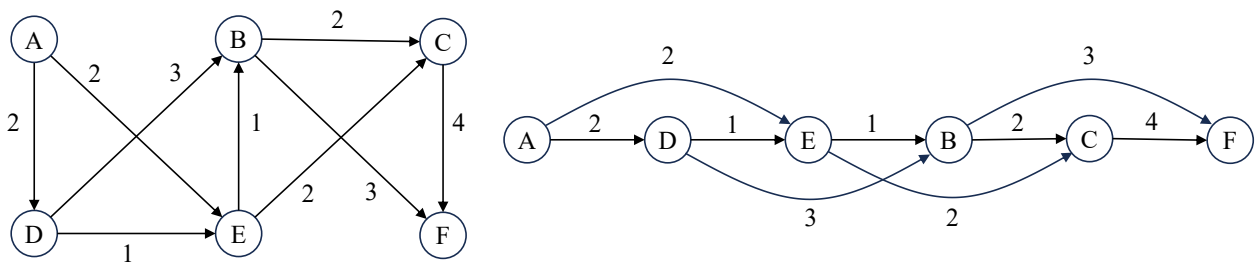


図 1 DAG とその線形化

有向非巡回グラフ (DAG) に対しては、最短パスを見つける問題は簡単に解けた。その理由は、頂点をトポロジカル順序に並べることができるためである (図 1 参照)。例えば頂点 A からの最短距離 (最短パスの長さ) を求めたいとき、頂点 C に対しては、頂点 E と B の二つの可能性を考えればよい。つまり、

$$\text{dist}(C) = \min\{\text{dist}(E) + 3, \text{dist}(B) + 2\}$$

が成り立つ。このような関係がすべての頂点に対し成り立つため、図 1 右側のトポロジカル順序に並べた頂点に対し、左から順に dist を定めることができる。したがって、初期頂点を s として、以下の手続きで計算できる。

```

1   for ( すべての頂点uについて ) dist(u) = ∞;
2   dist(s) = 0;
3   for ( 頂点 v をトポロジカル順序で選ぶ ) {
4       if ( v != s ) {
5           dist(v) = min{ dist(u) + w(u,v) : すべての辺(u,v) }
6       }
7   }
```

このアルゴリズムでは、各頂点 v に対する部分問題「始点 s から頂点 v までの最短距離を求める」を考え、依存関係に従って順番に解いている。頂点 v に対する部分問題は、 v へ入る辺 (u, v) の頂点 u に対応する部分問題の解を用いて解くことができる。

この計算方法は極めて一般的であり、各頂点の値を、その祖先頂点の値に基づいて順次計算している。上記では和の最小値をとることで最短パスを求めたが、最大値をとれば最長パスを計算できる。辺重みが正であるときには、和の代わりに積を用いて、辺重みの積が最小のパスを計算することもできる。

動的計画法は、部分問題を見つけ、それを小さい順に一つ一つ解き、小さい部分問題の解を使って大きい部分問題の解を見つけ、最終的にもとの問題を解くという方法論である。先程のような DAG は明示的

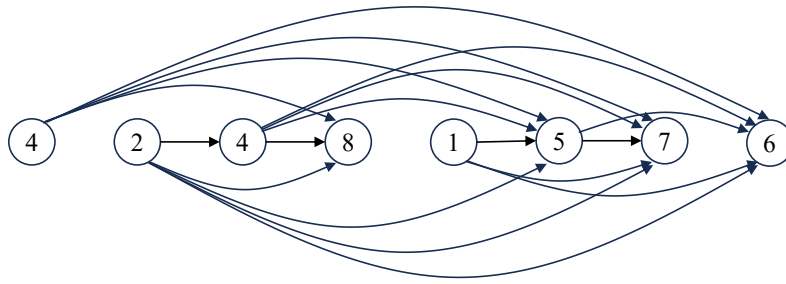


図2 増加部分列の DAG

には与えられないが、動的計画法には *DAG* が隠れている。部分問題が頂点に対応し、部分問題 B を解くために部分問題 A を解く必要があるとき、頂点 A から B への辺を張る。いくつか具体例を見てみよう。

2 最長増加部分列

最長増加部分列 (longest increasing subsequence) 問題では、数列 $a_1, a_2, \dots, a_n$ が与えられる。この数列の部分列 $a_{i_1}, a_{i_2}, \dots, a_{i_k}$ とは、 $i_1 < i_2 < \dots < i_k$ を満たすものであり、増加列とは各要素が真に大きくなる数列のことである。問題は、増加部分列として最長のものを見つけることである。例えば、4, 2, 4, 8, 1, 5, 7, 6 が与えられたとき、最長増加部分列の一つは 2, 4, 5, 7 である。

この問題の DAG は図 2 の通りである。数列の各要素が頂点であり、増加する二要素に対し辺を張る。つまり、数列 $a_1, \dots, a_n$ の要素 a_i が頂点 i に対応し、 $i < j$ かつ $a_i < a_j$ であるときに辺 (i, j) が存在する。各辺 (i, j) は $i < j$ を満たすため、DAG である。増加部分列はこのグラフにおけるパスに対応するため、最長増加部分列を見つける問題は、パスに含まれる頂点数が最大のパスを見つける問題である。

アルゴリズムは以下の通りである。

```

1  for ( j = 1, 2, ..., n ) {
2      L(j) = 1 + max{ L(i) : i < j かつ a[i] < a[j] };
3  }
4  return max{ L(1), ..., L(n) }
```

ここで、 $L(j)$ は a_j で終わる増加部分列の長さであり、候補となる辺がない場合の最大値は 0 とする。したがって、最長増加部分列の長さは上記の返り値である。このアルゴリズムは最長増加部分列の長さだけを求めている。最長増加部分列自体を求めたい場合は、各 $L(j)$ を計算するときに、頂点 j に至る直前の頂点を $\text{prev}(j)$ として記憶しておけばよい。

各 j について、 $i < j$ を満たすすべての i を調べれば $L(j)$ を $O(n)$ 時間で計算できる。したがって、このアルゴリズムの実行時間は $O(n^2)$ である。また、 prev をたどることで、最長増加部分列自体を $O(n)$ 時間で復元できる。

3 編集距離

文字列同士の近さを測りたい。例えば、「東京科学大学」は、「東京工業大学」と「東京医科歯科大学」のどちらに近いだろうか。文字の削除・挿入・置換をコストと考え、ある文字列を別の文字列に変換するのに必要な最小コストのことを**編集距離** (edit distance) と呼ぶ。いくつかの定義があるが、ここでは 1 文字の削除、1 文字の挿入、1 文字の置換をそれぞれコスト 1 と数える定義を採用する。

例えば、東京科学大学を編集（挿入・削除・置換）して東京工業大学に変換することを考える。図 3 の

東京 科学 大学 東京 科 学 大 学

東京 工 業 大 学 東京 工 業 大 学

図3 編集のコスト

左側だと、「工」を挿入し、「科」を「業」に置換し、「学」を削除すればよいので、コストは3である。一方、右側は、「科」を「工」に置換し、「業大」を挿入し、「大学」を削除するので、コストは5である。このようなコストの最小値が編集距離である。コスト最小なのは「科学」を「工業」に置換することであり、編集距離は2である。また、東京科学大学と東京医科歯科大学の編集距離は3、東京理科大学との編集距離は2であることがわかる(図4参照)。

東京 科学 大学 東京 科学 大学 東京 科学 大学

東京 工業 大学 東京 医 科 歯 科 大 学 東京 理 科 大 学

図4 編集距離

離は2であることがわかる(図4参照)。

これらは比較的簡単に距離が求まったが、少し複雑になると編集距離を求めることは難しい。例えば、「ALGORITHM」と「LOGARITHM」の編集距離はいくつだろうか。また、「ALGORITHM」と「PROGRAM」の編集距離はいくつだろうか。ここで登場するのが動的計画法である。

動的計画法を使って問題を解くとき、最も難しいのは「部分問題」を見つけることである。文字列 $a[1 \dots m]$ と $b[1 \dots n]$ が与えられ、これらの編集距離を計算したい。ここで、各文字列の先頭部分の編集距離を求める問題を考える。つまり、 $1 \leq i \leq m, 1 \leq j \leq n$ に対し、

$$E(i, j) = a[1 \dots i] \text{ と } b[1 \dots j] \text{ の編集距離}$$

と定める。こうすれば、最終的に解きたいのは $E(m, n)$ を求めることである。また、 $i = 0$ や $j = 0$ に対しても $E(i, j)$ を定義する。空文字列を別の文字列に変換するには、すべての文字を挿入または削除すればよいので、

$$E(0, j) = j, \quad E(i, 0) = i$$

である。次に、各 $E(i, j)$ を、より小さい部分問題の解から求める方法を考える。文字列 $a[1 \dots i]$ と $b[1 \dots j]$ の右端を見ると、図5の三通りが存在する。

$a[i]$ $a[i]$

または または

$b[j]$ $b[j]$

図5 編集操作の三つの場合

ここでは、 $a[1 \dots i]$ を編集して $b[1 \dots j]$ に変換することを考えると、左端は $a[i]$ を削除すること、中央は $b[j]$ を挿入することに相当する。右端は、 $a[i] = b[j]$ なら編集なし、 $a[i] \neq b[j]$ なら置換に相当する。もし左端の場合、 $E(i-1, j)$ が求まれば、それに1を足したコストで実現できる。中央の場合は、 $E(i, j-1)$

に 1 を足したコストで実現できる。右端の場合は、 $E(i-1, j-1)$ を求め、右端が $a[i] = b[j]$ ならコストを足さず $E(i-1, j-1)$ のまま、 $a[i] \neq b[j]$ なら $E(i-1, j-1)$ に 1 を足せばよい。したがって、以下の関係が成り立つ。

$$E(i, j) = \min\{1 + E(i-1, j), 1 + E(i, j-1), \text{diff}(i, j) + E(i-1, j-1)\}$$

ここで、 $\text{diff}(i, j)$ は、 $a[i] = b[j]$ なら 0、それ以外は 1 をとる。上記関係式に従い、 $E(i, j)$ の値をすべての i, j について求めればよい。

図 6 の右側は、ALGORITHM と PROGRAM の編集距離の部分問題 $E(i, j)$ の計算結果である。 $i = 0, j = 0$ から値が定まり、図の左側の手順で最小値を計算すればよい。表を一行ずつ計算していくため、計算量は $O(nm)$ である。

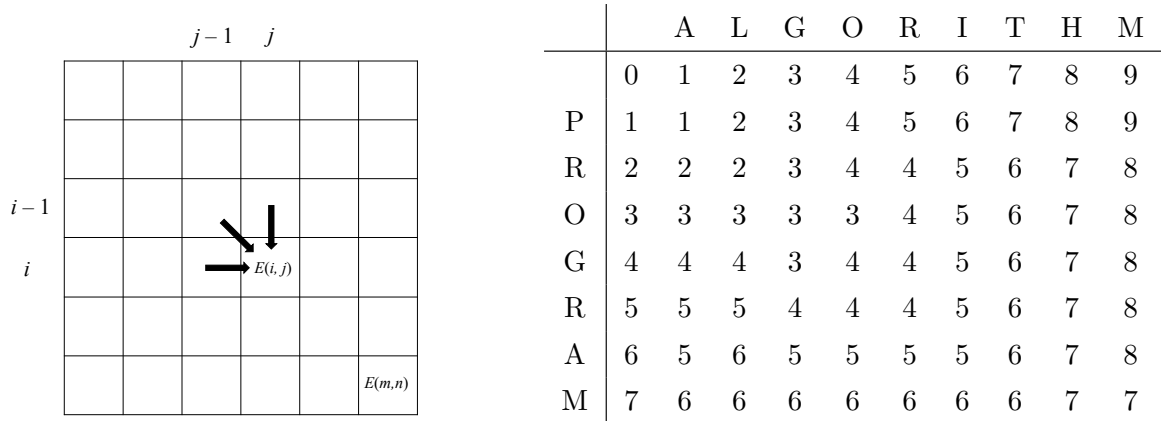


図 6 部分問題 $E(i, j)$ の計算

編集距離問題における DAG は、各部分問題 $E(i, j)$ が頂点に対応し、それを解くために必要な頂点から辺を張る。挿入と削除に対応する辺の重みは 1 であり、置換または一致に対応する斜めの辺の重みは $\text{diff}(i, j)$ である。この DAG において最短パスを見つけることが編集距離の計算に対応する。図 7 は、ALGORITHM と PROGRAM に対する DAG であり、重み 0 の辺は点線で表している。

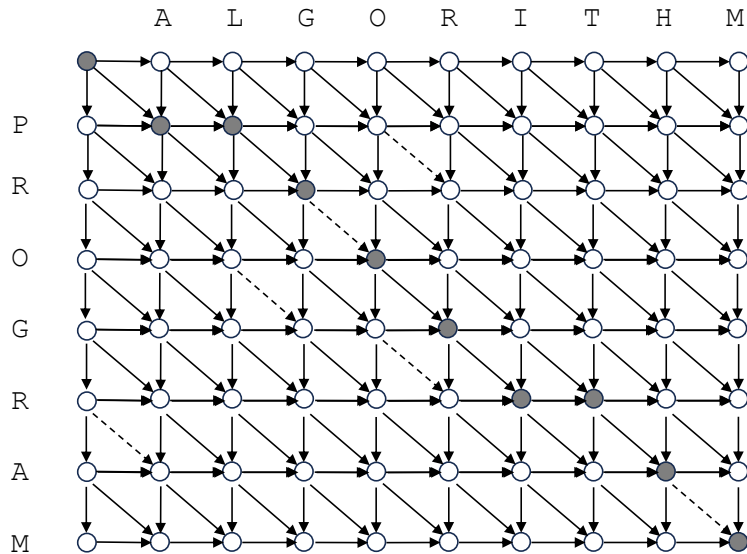


図 7 編集距離の DAG

このグラフから分かる通り、各編集操作の挿入・削除・置換に異なるコスト（重み）を割り当て、その場合の最短パスを計算することもできる。

4 ナップサック問題

宝石店に強盗に入ったときを考える。カバン（ナップサックだとする）を持ってきたが、詰められる荷物の重さは W g までという制限がある。店には n 種類の商品があり、それぞれ重さが $w_1, \dots, w_n$ であり、価値は $v_1, \dots, v_n$ である。このとき、重さ制限の中で価値を最大にする詰め方を知りたい。例えば、 $W = 700$ であり、以下の表の通りのとき、どうすればよいだろうか。

商品	重さ	価値
1	400 g	70 万円
2	300 g	40 万円
3	250 g	55 万円
4	150 g	25 万円

問題として、(1) 各商品の数に制限がない場合と、(2) 各商品が一つずつしかない場合の二つを考える。(1) の場合、商品 3 を 2 つ、商品 4 を 1 つ選べば価値 135 万円である。(2) の場合、商品 1 と 3 を選べば価値 125 万円である。

いずれの問題についても、入力長に関する多項式時間アルゴリズムは知られていないが、動的計画法を用いることで $O(nW)$ 時間で解くことができる。この実行時間は数値 W そのものに関しては多項式であるが、 W の入力長 $\Theta(\log W)$ に関しては指数関数的であり、入力全体の長さに関する多項式時間ではない。このような実行時間を擬多項式時間 (*pseudo-polynomial time*) という。

4.1 商品数無制限の場合

動的計画法にするため、部分問題を考える必要がある。より小さな問題として、重さ制限 W を小さくする方法と、商品の数を小さくする方法が考えられる。最初の方法のため、

$$V(w) = \text{重さ制限 } w \text{ で達成可能な価値の最大値}$$

とおく。もし $V(w)$ を達成する解に商品 i が含まれているとき、それを取り除いたものは、 $V(w - w_i)$ の最適解になるはずである。このとき、 $V(w) = V(w - w_i) + v_i$ であることがわかる。ただし、どの i が解であるかわからないため、すべてを試す。つまり、

$$V(w) = \max \{0, \max \{V(w - w_i) + v_i : w_i \leq w\}\}$$

である。空集合の価値は 0 とする。これにもとづいたアルゴリズムは以下の通りである。

```
1  V(0) = 0;
2  for ( w = 1, 2, ..., W ) {
3      V(w) = 0;
4      for ( w[i] <= w を満たすすべての i ) {
5          V(w) = max{ V(w), V(w - w[i]) + v[i] };
6      }
7  }
8  return V(W);
```

要素数 $W + 1$ の表 V の値を小さい順に埋めており、各繰り返しの計算時間は $O(n)$ であるため、実行時間は $O(nW)$ である。

上記に対応する DAG も存在する。ナップサック問題は、最長パスを求める問題に対応する。

4.2 商品一つずつの場合

各商品をつつまでしか選べない場合、前の戦略は使えない。 $V(w - w_i)$ に商品 i が含まれているかわからないためである。そこで、部分問題をつくるため、使う商品の数を制限する。 $0 \leq j \leq n$ に対し、

$V(w, j) =$ 重さ制限 w のとき商品 $1, \dots, j$ を使って達成可能な価値の最大値

とおく。求めたい答えは $V(W, n)$ である。部分問題 $V(w, j)$ を、より小さな部分問題で表したい。商品 j を使って最適な価値を達成する場合と、そうでない場合に分けられる。したがって、

$$V(w, j) = \begin{cases} V(w, j-1), & w_j > w, \\ \max\{V(w - w_j, j-1) + v_j, V(w, j-1)\}, & w_j \leq w \end{cases}$$

が成り立つ。アルゴリズムとしては、 $(W + 1) \times (n + 1)$ の 2 次元の表 V を小さい順に埋めればよい。表の各要素の計算は $O(1)$ ができるため、合計実行時間は $O(nW)$ である。アルゴリズムは以下の通り。

```
1  for ( j = 0, ..., n ) V(0, j) = 0;
2  for ( w = 0, ..., W ) V(w, 0) = 0;
3  for ( j = 1, ..., n ) {
4      for ( w = 1, 2, ..., W ) {
5          if ( w[j] > w ) V(w, j) = V(w, j-1);
6          else V(w, j) = max{ V(w - w[j], j-1) + v[j], V(w, j-1) };
7      }
8  }
9  return V(W, n);
```

5 メモ化

動的計画法の実装には、大きく分けて二つの方法がある。一つは、小さい部分問題から順に表を埋める **ボトムアップ** (bottom-up) の方法である。これまで見てきたアルゴリズムは、主にこの方法を用いていた。

もう一つは、もとの問題から再帰的に必要な部分問題と呼び出す **トップダウン** (top-down) の方法である。動的計画法は、トップダウンにもボトムアップにも実装できることが多い。トップダウンでは、漸化式を再帰関数としてほぼそのまま記述できるが、そのままでは同じ部分問題を何度も計算してしまう（第 1 回で扱ったフィボナッチ数の例）。そこで、一度計算した値を記憶して再利用する。この技法を **メモ化** (memoization) という。

商品数無制限のナップサック問題の場合、すでに計算した値 $V(w)$ を覚えるため、ハッシュ表を使うことができる。 $V(w)$ を計算する必要があるとき、表にすでに値が記録されているかを確認する。

```
1  int knapsack( int w ) {
2      if ( w == 0 ) return 0;
3      if ( w がハッシュ表にある ) return V(w);
4      V(w) = 0;
5
6      for ( w[i] <= w を満たすすべての i ) {
```

```

7      V(w) = max{ V(w), knapsack(w - w[i]) + v[i] };
8  }
9      ハッシュ表に、鍵 w の要素 V(w) を入れる;
10     return V(w);
11 }

```

実行時間は、最悪の場合には $W+1$ 個の部分問題を計算し、各部分問題に $O(n)$ 時間かかるため、 $O(nW)$ である。この計算量は動的計画法と変わらないが、 $O(\cdot)$ に隠れた定数は大きくなってしまう。しかし、このメモ化が有効に働く場合もある。例えば、すべての商品重量と重さ制限 W が 100 の倍数ならば、再帰呼出しで現れる重さ w も 100 の倍数に限られる。メモ化では、実際に必要となった部分問題だけを計算するため、それ以外の w について表を埋める必要がない。

6 連鎖行列積

行列積 $A \times B \times C \times D$ を計算したいとする。行列 A, B, C, D はそれぞれ $20 \times 50, 50 \times 1, 1 \times 10, 10 \times 100$ とする。行列積は可換 (commutative) ではないが、結合則 (associative law) が成り立つため、積の値は括弧の付け方に依存しない。つまり、 $A \times (B \times C) = (A \times B) \times C$ である。しかし、括弧の組み合わせ方によって計算の順番が異なり、必要な演算回数も異なる。

一般に、 $m \times n$ 行列と $n \times k$ 行列の積は、 mnk 回の掛け算を行う。これを考慮して計算コストを算出すると、大きな違いがあることがわかる。

括弧の組み合わせ	コストの計算	コスト
$A \times ((B \times C) \times D)$	$50 \cdot 1 \cdot 10 + 50 \cdot 10 \cdot 100 + 20 \cdot 50 \cdot 100$	150,500
$(A \times (B \times C)) \times D$	$50 \cdot 1 \cdot 10 + 20 \cdot 50 \cdot 10 + 20 \cdot 10 \cdot 100$	30,500
$(A \times B) \times (C \times D)$	$20 \cdot 50 \cdot 1 + 1 \cdot 10 \cdot 100 + 20 \cdot 1 \cdot 100$	4,000

問題としては、行列 $A_1 \times A_2 \times \cdots \times A_n$ を計算するためにコストが最小の計算順を見つけたい。ここで、 $A_1, A_2, \dots, A_n$ はそれぞれ $m_0 \times m_1, m_1 \times m_2, \dots, m_{n-1} \times m_n$ 行列括弧の対応は全二分木 (full binary tree) で表現できるが (図 8 参照)、葉の数が n のとき、 n に関して指数関数的な数の組み合わせが存在する。したがって、すべての場合を試すことはできず、動的計画法を用いることにする。

図 8 のような二分木で表現するとわかるが、ある木が最適コストを達成するとき、その部分木も最適コストを達成しているはずである。したがって、

$$C(i, j) = A_i \times A_{i+1} \times \cdots \times A_j \text{ の行列積の最小コスト}$$

と定めると、この部分問題のサイズは $|j-i|$ であり、 $i=j$ のときは行列積を行う必要がないため、 $C(i, i) = 0$ である。 $i < j$ のときに、 $C(i, j)$ の最適な部分木を考えると、ある $i \leq k < j$ が存在し、 $A_i \times \cdots \times A_k$ と $A_{k+1} \times \cdots \times A_j$ という二つに分けられ、 $C(i, j)$ のコストは、 $C(i, k) + C(k+1, j) + m_{i-1} \cdot m_k \cdot m_j$ である。最小コストの k を見つければよく、

$$C(i, j) = \min_{i \leq k < j} \{C(i, k) + C(k+1, j) + m_{i-1} \cdot m_k \cdot m_j\}$$

が成り立つ。アルゴリズムは以下の通り。変数 s は部分問題サイズを表している。

```

1  for ( i = 1, ..., n ) C(i,i) = 0;
2  for ( s = 1, ..., n-1 ) {
3      for ( i = 1, ..., n-s ) {

```

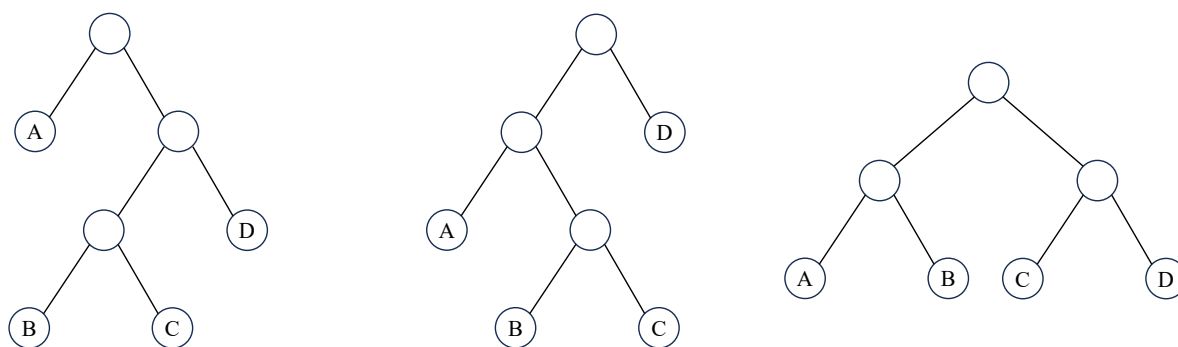


図8 全二分木による括弧の表現

```

4      j = i + s;
5      C(i,j) = min{ C(i,k) + C(k+1,j) + m[i-1]*m[k]*m[j] : i <= k < j }
6  }
7  }
8  return C(1,n);

```

部分問題 $C(i,j)$ は $O(n^2)$ 個あり、各部分問題の計算では $O(n)$ 個の k を調べるため、実行時間は $O(n^3)$ である。

7 最短パス

最短パス問題は、グラフ $G = (V, E)$ と頂点 $s, t \in V$ が与えられたとき、 s - t 間の最短パスを求める問題である。

7.1 制約付き最短パス

ここでは、パスの長さに制限がある場合を考える。つまり、 k が与えられたときに、 k 本以下の辺を辿るパスの中での最短パスを求めるという問題である。

通常のダイクストラ法では、辿った辺の本数を状態として保持しないため、そのままでは適用できない。

動的計画法では、辿ってきた辺の本数を考慮して部分問題を設定することができる。頂点 s から頂点 v までちょうど i 本の辺を辿るパスの最短距離を $\text{dist}(v, i)$ と表す。初期値を

$$\text{dist}(s, 0) = 0, \quad \text{dist}(v, 0) = \infty \quad (v \neq s)$$

とし、

$$\text{dist}(v, i) = \min_{(u,v) \in E} \{ \text{dist}(u, i-1) + w(u, v) \}$$

と計算する。

7.2 全ペア最短パス

頂点 s - t 間だけでなく、すべての頂点ペアの最短パスを求めるにはどうすればよいだろうか。負重みを考慮してベルマン-フォード法を各頂点に対して実行すると、実行時間は $O(|V|^2|E|)$ である。ここでは、動的計画法を利用して実行時間 $O(|V|^3)$ を達成するワーシャル-フロイド法 (Floyd-Warshall algorithm) を見てみよう。

まず、良い部分問題を考える必要がある。素朴に、対象の頂点を順に増やしていく方法だと、 $O(|V|^2|E|)$ になってしまう。そこで、利用できる中間ノードを順に増やしていくという方法をとる。頂点集合 $V = \{1, 2, \dots, n\}$ に対し、

$\text{dist}(i, j, k) =$ 中間ノードとして $\{1, \dots, k\}$ だけが使えるときの頂点 i から j への距離

と定める。初期値 $\text{dist}(i, j, 0)$ は、辺 (i, j) があればその重み、なければ ∞ である。中間頂点として頂点 k を使わない場合と使う場合に分けると、

$$\text{dist}(i, j, k) = \min\{\text{dist}(i, j, k-1), \text{dist}(i, k, k-1) + \text{dist}(k, j, k-1)\}$$

が成り立つ。

中間ノードとして頂点 k の利用が許されたとき、どのような計算をする必要があるだろうか。頂点 i から j への最短パスにおいて、頂点 k は一度しか登場しないことがわかる。負重みの閉路がないと仮定しているからである。したがって、頂点 k を最短パスに含めるのは、

$$\text{dist}(i, k, k-1) + \text{dist}(k, j, k-1) < \text{dist}(i, j, k-1)$$

を満たすときであることがわかる。アルゴリズムは以下の通りであり、計算量は $O(|V|^3)$ である。

```

1   for ( i = 1, ..., n )
2       for ( j = 1, ..., n ) dist(i,j,0) = ∞;
3   for ( i = 1, ..., n ) dist(i,i,0) = 0;
4   for ( すべての辺 (i,j) に対し ) dist(i,j,0) = min{ dist(i,j,0), w(i,j) };
5
6   for ( k = 1, ..., n )
7       for ( i = 1, ..., n )
8           for ( j = 1, ..., n )
9               dist(i,j,k) = min{ dist(i,j,k-1), dist(i,k,k-1) + dist(k,j,k-1) };
```

7.3 巡回セールスマン問題

やり手のセールスマンは、訪問先同士の最短パスがわかるだけでは満足できない。会社から、すべての訪問先を1度ずつ訪問して会社に戻ってくる、この条件下で最短の経路を知りたい。問題としては、訪問先 $\{1, 2, \dots, n\}$ （自分の会社は1）と各訪問先 $i-j$ 間の距離 $D(i, j)$ が与えられたとき、1から出発し、ほかの各訪問先をちょうど一度ずつ訪れて、最後に1へ戻る巡回路のうち、距離が最小のものを求める。巡回セールスマン問題 (traveling salesperson problem, TSP) と呼ばれている。

もっとも素朴な方法は、すべての組み合わせ $(n-1)!$ 通りの距離を計算することであるが、計算量はかなり大きい。そこで、動的計画法を用いて高速化することを考える。ただし、多項式時間ではない。

部分問題として、訪問先の一部 $S \subseteq \{1, \dots, n\}$ （1は必ず含む）を用いてある $j \in S$ に訪問するまでの最短パスを考える。つまり、1を含む集合 $S \subseteq \{1, \dots, n\}$ と $j \in S$ に対し、

$C(S, j) =$ 1から出発し、 S の各訪問先をちょうど一度ずつ訪れ、 j で終わるパスの最小距離

と定める。初期条件は $C(\{1\}, 1) = 0$ である。また、 $|S| > 1$ のときは途中で1に戻ることを許さないため、

$$C(S, 1) = \infty$$

とする。最後に利用する辺として (i, j) を採用するか否かを検討すれば、 $j \neq 1$ のとき、 j の直前に訪れた頂点を i とすると、

$$C(S, j) = \min_{i \in S \setminus \{j\}} \{C(S \setminus \{j\}, i) + D(i, j)\}$$

が成り立つ。すべての訪問先を訪れた後、最後の訪問先 j から 1 へ戻る必要がある。したがって、求める答えは

$$\min_{j \neq 1} \{C(\{1, \dots, n\}, j) + D(j, 1)\}$$

である。

部分問題は $O(n2^n)$ 個あり、一つの計算に $O(n)$ かかるため、合計計算量は $O(n^2 2^n)$ である。

8 動的計画法の演習問題

演習問題 8.1. リスト S の隣接部分列 (contiguous subsequence) とは、 S から連続した要素を選んだものである。例えば、 $S = [5, 15, -30, 10, -5, 40, 10]$ であれば、 $[15, -30, 10]$ は隣接部分列であるが、 $[5, 15, 40]$ は隣接部分列ではない。リストの要素 $a_1, \dots, a_n$ が与えられたとき、隣接部分列の中で、要素の和が最大になるもの（長さ 0 の部分列の和は 0 とする）を見つけるアルゴリズムを与えよ。上記の S に対しては、 $[10, -5, 40, 10]$ を出力すれば良い。（ヒント：各 $j \in \{1, \dots, n\}$ に対し、 j 番目の要素で終わる隣接部分列を考えてみよう。）

演習問題 8.2. 車で旅行に出かける。目的地のホテルまで n 個のホテルが存在し、各ホテル i はここから距離 a_i (km) 離れていて、 $a_1 < a_2 < \dots < a_n$ である。目的地であるホテル n で停止する必要があるが、その他のホテルは停止の必要はない。1 日に 300 km 移動するのが理想的であり、1 日に x km 移動したとき、 $(300 - x)^2$ のペナルティが発生する。このとき、ペナルティを最小にするホテルの停止方法を求める効率的なアルゴリズムを与えよ。

演習問題 8.3. あるドーナツのチェーン店が高速道路のサービスエリアで出店する計画をしている。入口から距離 $d_1, d_2, \dots, d_n$ の位置に n 個のサービスエリアがある ($d_1 < \dots < d_n$)。以下の制約を考慮する必要がある。

- 各サービスエリアでは 1 店舗までしか出店できず、 i 番目のサービスエリアで出店すれば利益 p_i が見込める。
- 各店舗は少なくとも k km 離れた場所に出店しなければならない。

この制約下で、見込み利益の最大値を求める効率的なアルゴリズムを与えよ。

演習問題 8.4. 4 行 n 列のボードがあり、各マスに数字が書かれてある。 $2n$ 個の石があり、それらをボードの各マスに置くことができ、置いた石の数字の合計を最大化したい（各マスには石は 1 個までしか置けない）。1 つ制約があり、縦と横に隣接するマスに石を置くことはできない。

- (a) 各列に置くことが可能な石のパターン（隣の列は考慮しなくて良い）の数を求め、そのパターンを書き下せ。

2 つの列パターンが両立可能 (compatible) とは、それらが隣ある列となっても制約に違反しないときである。最初の k 列 ($1 \leq k \leq n$) からなる部分問題を考える。各部分問題における最後の列のパターンをタイプと呼ぶ。

- (b) 両立可能性とタイプという概念を用いて、計算量 $O(n)$ で最適な石の配置を求めるアルゴリズムを動的計画法を用いて設計せよ。

演習問題 8.5. 記号 a, b, c に対し掛け算を以下の表で定義する。例えば $ab = b, bc = a$ であり、この掛け算は、結合性 (associativity) や可換性 (commutativity) は満たさない。与えられた記号列に対し、括弧をつ

	a	b	c
a	b	b	a
b	c	b	a
c	a	c	c

けることで掛け算結果を a にすることができるか否かを判定する効率的なアルゴリズムを与えよ。例えば、入力 $bbbbaac$ に対しては、 $((b(bb))(ba))c = a$ であるため、出力は YES である。

演習問題 8.6. 部分列が回文 (*palindrome*) であるとは、逆順にしても同じ文字列になるときである。文字列

$A, C, G, T, G, T, C, A, A, A, A, T, C, G$

の中には、 A, C, G, C, A や A, A, A, A などの回文が含まれる。文字列 $x[1..n]$ を受け取り、最長の回文を返す計算量 $O(n^2)$ のアルゴリズムを与えよ。

演習問題 8.7. 整数 n, k ならびに $p_1, p_2, \dots, p_n \in [0, 1]$ が与えられたとき、偏りのあるコインを n 枚独立に投げたときに正確に k 回表が出る確率を求めたい。ここで、 p_i は i 番目のコインが表になる確率である。この問題を計算量 $O(n^2)$ で解くアルゴリズムを与えよ。ただし、範囲 $[0, 1]$ 内の数の加算と乗算は $O(1)$ 時間でできるとする。

演習問題 8.8. n 頂点の凸多角形 P が与えられる（各頂点の (x, y) 座標が与えられる）。 P の三角形分割 (triangulation) とは、 P の中に $n - 3$ 本の対角線を引き、それらが交差しない（端点で交差することはある）ようにすることである。三角形分割により、 P は $n - 2$ 個の互いに交わらない三角形に分割される。三角形分割のコストは、対角線の長さの合計である。最小コストの三角形分割を求める効率的なアルゴリズムを与えよ。（ヒント： P の頂点を任意の点から時計回りに $1, 2, \dots, n$ とラベルをつけると、 $1 \leq i, j \leq n$ に対し、部分問題 $A(i, j)$ を $i, i + 1, \dots, j$ で構成される多角形の最小コスト三角形分割と定めてみよう。）

演習問題 8.9. 以下のゲームを考える。ディーラーがカード列 $s_1, s_2, \dots, s_n$ を表にして並べる。各 s_i には値 v_i が割り当てられている。2 人のプレイヤーは、カード列の最初または最後だけを選んで手元に加え、合計値が大きい方が勝ちである。（例えば、カードは異なる額面の紙幣と考えればよい。）

- (a) カード列として、最初のプレイヤーがより大きな値のカードを選ぶことが最適でない例を示せ。つまり、単純な貪欲法が最適でないことを示せ。
- (b) 最初のプレイヤーの最適戦略を計算する計算量 $O(n^2)$ のアルゴリズムを与えよ。初期カード列に対して、アルゴリズムは計算量 $O(n^2)$ の事前計算である情報を計算し、その後各手番では、その情報を参照することで、最初のプレイヤーの動きを計算量 $O(1)$ で行えるようにすること。

演習問題 8.10. 2 つのチーム A と B が試合を行っており、どちらが先に n ゲームを先取するかを競っている。 A と B は同等の実力であり、どの試合も勝率は 50% ずつである。すでに $i + j$ ゲームが行われ、 A が i ゲーム、 B が j ゲームを取っている。このとき、 A が試合に勝つ確率を計算する効率的なアルゴリズムを与えよ。例えば、 $i = n - 1$ 、 $j = n - 3$ のとき、 A が試合に勝つ確率は $7/8$ である。なぜならば、 A は次の 3 ゲームのうち 1 つでも勝てばよいからである。

演習問題 8.11. 額面が $x_1, x_2, \dots, x_n$ のコインが無限にあり、コインを組み合わせる価値の合計が v になる組み合わせを見つけたい。例えば、 $x_1 = 5, x_2 = 10$ のとき、 $v = 15$ は可能であるが、 $v = 12$ は不可能である。入力 $x_1, \dots, x_n, v$ が与えられたとき、額面 $x_1, \dots, x_n$ のコインを使って価値 v を作ることが可能

かどうかを判定する計算量 $O(nv)$ の動的計画法アルゴリズムを与えよ.

演習問題 8.12. 前問の変形を考える. 入力 $x_1, x_2, \dots, x_n, v$ が与えられたとき, 額面 $x_1, \dots, x_n$ のコインを使って価値 v を作ることが可能かどうかを判定するが, 各コインは一度までしか使えないものとする. 例えば, 額面が 1, 5, 10, 20 のとき, $16 = 1 + 5 + 10$ や $31 = 1 + 10 + 20$ は可能であるが, 40 は不可能である. この問題を解く計算量 $O(nv)$ のアルゴリズムを与えよ.

演習問題 8.13. さらに別の変形を考える. 入力として $x_1, x_2, \dots, x_n, v$ ならびに k が与えられ, 額面 $x_1, \dots, x_n$ のコインを高々 k 枚使って価値 v を作ることが可能かどうかを判定したい. 例えば, 額面が 5, 10 のとき, $k = 6$ ならば 55 は可能であるが, 65 は不可能である. この問題を解く動的計画法アルゴリズムを与えよ.

演習問題 8.14. 以下の三分割問題を考える. 非負整数 $a_1, a_2, \dots, a_n$ が与えられたとき, それらを互いに交わらない (disjoint) 部分集合 I, J, K に分割して,

$$\sum_{i \in I} a_i = \sum_{j \in J} a_j = \sum_{k \in K} a_k = \frac{1}{3} \sum_{i=1}^n a_i$$

を満たすことができるかどうか判定したい. 例えば, $(1, 2, 3, 4, 4, 5, 8)$ に対しては, $(1, 8), (4, 5), (2, 3, 4)$ のように分割できるが, $(2, 2, 3, 5)$ に対しては分割できない. この問題を解く動的計画法アルゴリズムを与えよ. 計算量は n ならびに $\sum_{i=1}^n a_i$ について多項式であること.

演習問題 8.15. 新しい遺伝子が発見されたとき, その機能を理解するための標準的な方法は, 既知の遺伝子データベースを参照し, 類似性の高い遺伝子を見つけることである. 2つの遺伝子の類似性は, それらの整列 (alignment) の程度で測る. 1つの遺伝子はアルファベット $\Sigma = \{A, C, G, T\}$ 上の文字列と考える. 例えば, 遺伝子 $x = ATGCC$ と $y = TACGCA$ の整列は, 以下のように書ける.

$$\begin{array}{cccccc} x: & - & A & T & - & G & C & C \\ y: & T & A & - & C & G & C & A \end{array}$$

ここで, $-$ は空白 (gap) を表す. 文字列の各文字は順番に出現し, 各列にはいずれかの文字列の文字が含まれる. 整列のスコアは, $(|\Sigma| + 1) \times (|\Sigma| + 1)$ のスコア行列 δ で決まる. 上記整列のスコアは,

$$\delta(-, A) + \delta(A, A) + \delta(T, -) + \delta(-, C) + \delta(G, G) + \delta(C, C) + \delta(C, A)$$

である.

文字列 $x[1..n]$ と $y[1..m]$, スコア行列 δ を入力とし, 最高スコアの整列を返す動的計画法アルゴリズムを与えよ. 計算量は $O(nm)$ とする.